

UNIVERSITY OF SCIENCE VNU-HCM
FACULTY OF INFORMATION TECHNOLOGY
ADVANCED PROGRAM IN COMPUTER SCIENCE



???

Report

Students:

Tran Canh Anh Tuan 24125048
Le Dat Phuc An 24125052
Vong Chi Van 24125041
Pham Nguyen Minh Quan
24125041

Lecturers:

Nguyen Ngoc Toan, MSc.
Nguyen Ngoc Minh Chau, MSc.
Le Thi Nhan, PhD

2026

Summary

1	Introduction	3
1.1	Project Background	3
1.2	Project Objectives	3
1.3	Project Deliverables	3
2	Database Design	3
2.1	Business Requirement Analysis	3
2.1.1	Business Purpose	3
2.1.2	Actors	4
2.1.3	Core Entities	4
2.1.4	Business Rules	5
2.1.5	Requirement Summary	5
2.2	Conceptual Database Design	6
2.2.1	Entity Relationship Diagram	6
2.2.2	Design Decisions	6
2.2.3	Relationship Summary	6
2.3	Logical Database Design	6
2.3.1	Relational Schema	6
2.3.2	Relation Summary	7
2.3.3	Normalization	7
2.4	Database Validation	7
2.4.1	Requirement Traceability	7
2.4.2	Constraint Analysis	8
2.4.3	Design Limitations	8
3	Database Implementation & Testing	8
3.1	Database Implementation	8
3.1.1	Table Summary	9
3.1.2	Constraints	9
3.2	Sample Data	11
3.2.1	Test Coverage	12
3.3	Query Design	12
3.3.1	Query Summary	12
3.3.2	Representative Query Results	13
4	Conclusion	14
4.1	Project Summary	14
4.2	Key Achievements	15
4.3	Future Work	15

§1 Introduction

§1.1 Project Background

The School of Computer Science manages shared physical spaces (classrooms, laboratories, meeting rooms, auditoriums) for teaching, seminars, examinations, workshops, and student activities. Space requests are currently handled manually via email, phone, and spreadsheets, leading to booking conflicts, use of spaces under maintenance, and loss of usage history. This project develops a relational database system to replace the manual process.

§1.2 Project Objectives

- Centralize the management of all bookable campus spaces and their facilities.
- Prevent conflicting bookings and block reservations for unavailable spaces.
- Track maintenance issues from report through resolution.
- Preserve complete historical records of bookings and maintenance.
- Support operational decision-making with booking and usage data.

§1.3 Project Deliverables

Deliverable	Description
Business Requirement Analysis	Requirement extraction and analysis
ER Diagram	Conceptual database design
Logical Schema	Relational database design
SQL Implementation	Database implementation
Sample Data	Testing dataset
SQL Queries	Business queries

§2 Database Design

§2.1 Business Requirement Analysis

§2.1.1 Business Purpose

The system manages the booking and usage of shared campus spaces for the School of Computer Science, replacing a manual spreadsheet/email process with a structured database that handles booking requests, approvals, check-in/check-out, maintenance tracking, and historical reporting.

§2.1.2 Actors

Role	Description
Student	Submits booking requests for student activities; views own bookings
Lecturer	Submits booking requests for lectures and examinations
Teaching Assistant	Submits booking requests for workshops and tutorials
Facility Staff	Approves/rejects bookings, performs check-in/check-out, manages maintenance
Department Administrator	Views departmental booking history
Facility Manager	Oversees all spaces, bookings, maintenance, and facility statuses

§2.1.3 Core Entities

- USER
 - **PK:** user_id
 - **Attributes:** full_name, email, phone, role, department, account_status
- SPACE
 - **PK:** space_code
 - **Attributes:** space_name, space_type, building, floor, room_number, capacity, current_status, usage_policy
- FACILITY
 - **PK:** facility_id
 - **FK:** space_code
 - **Attributes:** facility_name, description
- BOOKING_REQUEST
 - **PK:** booking_id
 - **FK:** user_id, space_code
 - **Attributes:** requested_start_time, requested_end_time, purpose, expected_participants, booking_type, status
- BOOKING_APPROVAL
 - **PK:** approval_id
 - **FK:** booking_id (UNIQUE), decided_by_user_id
 - **Attributes:** decision_time, decision_note, rejection_reason
- USAGE_SESSION
 - **PK:** session_id
 - **FK:** booking_id (UNIQUE), checked_in_by_user_id, completed_by_user_id

- **Attributes:** `actual_start_time`, `actual_end_time`, `initial_condition`, `final_condition`, `usage_notes`
- **MAINTENANCE_RECORD**
 - **PK:** `maintenance_id`
 - **FK:** `space_code`, `reporter_user_id`, `assigned_staff_user_id`
 - **Attributes:** `problem_description`, `start_time`, `completion_time`, `status`, `result_note`

§2.1.4 Business Rules

1. No overlapping approved bookings for the same space.
2. Spaces with status `under_maintenance`, `temporarily_closed`, or `retired` cannot be booked.
3. Spaces with active maintenance records cannot be booked.
4. `requested_end_time` must be greater than `requested_start_time`.
5. `BOOKING_APPROVAL` is optional, at most one per `BOOKING_REQUEST` (1:0..1).
6. `USAGE_SESSION` is optional, at most one per `BOOKING_REQUEST` (1:0..1).
7. Rejected bookings must preserve `rejection_reason`.
8. Check-in must record `actual_start_time`, `checked_in_by_user_id`, `initial_condition`.
9. Completion must record `actual_end_time`, `completed_by_user_id`, `final_condition`, `usage_notes`.
10. Historical data must be preserved (no hard deletes).

§2.1.5 Requirement Summary

Requirement	Database Support
User management	<code>USER</code> with role and account status
Space inventory	<code>SPACE</code> with type, capacity, and status
Facility tracking	<code>FACILITY</code> linked to <code>SPACE</code> via FK
Booking workflow	<code>BOOKING_REQUEST</code> with status lifecycle
Approval management	<code>BOOKING_APPROVAL</code> (1:0..1 to request)
Usage tracking	<code>USAGE_SESSION</code> for check-in/check-out
Maintenance management	<code>MAINTENANCE_RECORD</code> from report to resolution
Conflict prevention	Triggers/application logic on approved bookings
History preservation	Soft-delete policy across all relations

§2.2 Conceptual Database Design

§2.2.1 Entity Relationship Diagram

The ERD was developed using Mermaid notation, modeling seven entities with their attributes, relationships, cardinalities, and participation constraints. The complete diagram is provided in the project deliverables (output/02-erd-design-G08.md).

§2.2.2 Design Decisions

- **Entity identification:** Seven entities map one-to-one to distinct real-world concepts—no merging or over-splitting.
- **Relationship design:** USER participates in multiple relationships with distinct semantic roles (requester, approver, check-in staff, reporter, assigned staff), modeled via separate foreign keys.
- **Cardinalities:** BOOKING_APPROVAL and USAGE_SESSION use 1:0..1 (not 1:N), reflecting at-most-one approval/session per booking.
- **Participation:** Total participation for dependent entities (e.g., every FACILITY must belong to a SPACE); partial participation where entities may exist independently.

§2.2.3 Relationship Summary

Relationship	Cardinality
Space contains Facility	1 : N
User submits Booking_Request	1 : N
Space receives Booking_Request	1 : N
Booking_Request has Booking_Approval	1 : 0..1
User performs Booking_Approval	1 : N
Booking_Request creates Usage_Session	1 : 0..1
User checks in Usage_Session	1 : N
User completes Usage_Session	1 : N
Space has Maintenance_Record	1 : N
User reports Maintenance_Record	1 : N
User assigned to Maintenance_Record	1 : N

§2.3 Logical Database Design

§2.3.1 Relational Schema

The ERD was converted into seven relations. The full schema definitions with data types are provided in the project deliverables (output/03-logical-design-G08.md).

§2.3.2 Relation Summary

Relation	PK	Foreign Keys	Candidate Keys
USERS	user_id	—	email
SPACES	space_code	—	(building, room_number)
FACILITY	facility_id	space_code	—
BOOKING_REQUEST	booking_id	user_id, space_code	—
BOOKING_APPROVAL	approval_id	booking_id (UNIQUE), decided_by_user_id	booking_id
USAGE_SESSION	session_id	booking_id (UNIQUE), checked_in_by_user_id, completed_by_user_id	booking_id
MAINTENANCE_RECORD	maintenance_id	space_code, reporter_user_id, as-signed_staff_user_id	—

All eleven foreign keys implement the ERD relationships. The UNIQUE constraint on booking_id in BOOKING_APPROVAL and USAGE_SESSION enforces the 1:0..1 cardinality.

§2.3.3 Normalization

All relations satisfy Third Normal Form (3NF):

- **1NF:** All attributes are atomic.
- **2NF:** No partial dependencies (all PKs are single-attribute).
- **3NF:** No transitive dependencies among non-key attributes.

§2.4 Database Validation

§2.4.1 Requirement Traceability

Every business requirement maps to a database component: user management to USERS, space inventory to SPACES, facility tracking to FACILITY, booking workflow to BOOKING_REQUEST, approval decisions to BOOKING_APPROVAL, usage tracking to USAGE_SESSION, and maintenance management to MAINTENANCE_RECORD.

§2.4.2 Constraint Analysis

Business Rule	Schema Constraint	Additional Enforcement
No overlapping bookings	—	Trigger / application logic
Unavailable spaces not bookable	CHECK on <code>current_status</code>	Application status check
Active maintenance blocks booking	—	Application logic
End time > start time	CHECK constraint	—
At most one approval per booking	UNIQUE on <code>booking_id</code>	—
At most one session per booking	UNIQUE on <code>booking_id</code>	—
Rejection reason preserved	—	Application logic
Valid user roles	CHECK on <code>role</code>	—
No hard deletes	—	Application policy

§2.4.3 Design Limitations

The following constraints cannot be expressed as single-table CHECK constraints and require triggers or application logic:

- **Overlapping booking prevention:** Requires querying existing approved bookings for time-range overlap detection.
- **Maintenance-based booking blocks:** Requires cross-table join between `BOOKING_REQUEST` and `MAINTENANCE_RECORD`.
- **Conditional NOT NULL:** Fields like `rejection_reason` and completion fields are conditionally required based on booking status.

§3 Database Implementation & Testing

§3.1 Database Implementation

The logical schema was implemented as a SQL Server database using T-SQL DDL statements. The complete implementation is provided in the project deliverables (`output/05-db-definition-`

§3.1.1 Table Summary

Table	Purpose
USERS	Stores all system users with role-based access (students, lecturers, TAs, staff, administrators, managers)
SPACES	Stores shared campus spaces with type, location, capacity, and availability status
FACILITY	Stores equipment and facilities installed in each space (projectors, computers, whiteboards, etc.)
BOOKING_REQUEST	Records booking requests with time range, purpose, participant count, and lifecycle status
BOOKING_APPROVAL	Records approval or rejection decisions for booking requests (at most one per request)
USAGE_SESSION	Records actual check-in/check-out data for approved bookings (at most one per request)
MAINTENANCE_RECORD	Tracks maintenance issues from report through assignment to resolution

§3.1.2 Constraints

3.1.2.1 Primary Keys. All seven tables use single-attribute primary keys (`user_id`, `space_code`, `facility_id`, `booking_id`, `approval_id`, `session_id`, `maintenance_id`), ensuring 2NF compliance with no partial dependencies.

3.1.2.2 Foreign Keys. Eleven foreign keys implement all relationships from the ERD:

Child Table	FK Column	Parent Table
FACILITY	<code>space_code</code>	SPACES
BOOKING_REQUEST	<code>user_id</code>	USERS
BOOKING_REQUEST	<code>space_code</code>	SPACES
BOOKING_APPROVAL	<code>booking_id</code>	BOOKING_REQUEST
BOOKING_APPROVAL	<code>decided_by_user_id</code>	USERS
USAGE_SESSION	<code>booking_id</code>	BOOKING_REQUEST
USAGE_SESSION	<code>checked_in_by_user_id</code>	USERS
USAGE_SESSION	<code>completed_by_user_id</code>	USERS
MAINTENANCE_RECORD	<code>space_code</code>	SPACES
MAINTENANCE_RECORD	<code>reporter_user_id</code>	USERS
MAINTENANCE_RECORD	<code>assigned_staff_user_id</code>	USERS

3.1.2.3 CHECK Constraints. The following CHECK constraints enforce domain integrity at the database level:

- `requested_end_time > requested_start_time` — ensures valid booking time ranges
- `actual_end_time >= actual_start_time` (or NULL) — ensures valid session durations

- `capacity > 0` — prevents zero or negative space capacity
- `expected_participants > 0` — prevents zero or negative participant counts
- `floor >= 0` — implied by INT NOT NULL (no explicit CHECK, enforced by data type)
- `role` restricted to: `student`, `lecturer`, `teaching_assistant`, `facility_staff`, `department_administrator`, `facility_manager`
- `space_type` restricted to: `classroom`, `computer_laboratory`, `meeting_room`, `project_laboratory`, `auditorium`, `workspace`
- `booking_type` restricted to: `lecture`, `examination`, `seminar`, `workshop`, `meeting`, `student_activity`, `administrative_event`
- `current_status` (SPACES) restricted to: `available`, `in_use`, `under_maintenance`, `temporarily_closed`, `retired`
- `status` (BOOKING_REQUEST) restricted to: `pending`, `approved`, `rejected`, `cancelled`, `checked_in`, `completed`, `no_show`
- `status` (MAINTENANCE_RECORD) restricted to: `pending`, `in_progress`, `completed`, `cancelled`
- `account_status` restricted to: `active`, `inactive`, `suspended`

3.1.2.4 UNIQUE Constraints. Four UNIQUE constraints enforce candidate keys and 1:0..1 cardinalities:

- `USERS.email` — ensures no duplicate email addresses
- `SPACES(building, room_number)` — ensures no duplicate rooms within a building
- `BOOKING_APPROVAL.booking_id` — enforces at most one approval per booking request
- `USAGE_SESSION.booking_id` — enforces at most one usage session per booking request

3.1.2.5 DEFAULT Values. Default values provide sensible initial states for status columns:

- `account_status` defaults to `'active'`
- `current_status` defaults to `'available'`
- `status` (BOOKING_REQUEST) defaults to `'pending'`
- `status` (MAINTENANCE_RECORD) defaults to `'pending'`

3.1.2.6 Triggers. Three business rules require triggers or application logic because they involve cross-row or cross-table validation:

1. **Overlapping booking prevention:** Before inserting or updating a `BOOKING_REQUEST` with status `'approved'`, the trigger checks for existing approved bookings on the same space with overlapping time ranges and raises an error if a conflict is found.
2. **Space availability check:** Before inserting a `BOOKING_REQUEST`, the trigger verifies that the target space's `current_status` is not `'under_maintenance'`, `'temporarily_closed'`, or `'retired'`, and that no active maintenance records exist for the space.
3. **Rejection reason enforcement:** When a `BOOKING_REQUEST` is set to `'rejected'`, the trigger ensures that the corresponding `BOOKING_APPROVAL` record contains a non-NULL `rejection_reason`.

§3.2 Sample Data

The sample dataset populates all seven tables with realistic data covering the full range of business scenarios. The complete dataset is provided in the project deliverables (`output/06-sample-data-G08.sql`).

§3.2.1 Test Coverage

Scenario	Covered
7 booking statuses: Pending, Approved, Rejected, Cancelled, Checked In, Completed, No-show	B019/B028 (Pending), B016/B018/B020/B022/B026/B027/B029 (Approved), B024/B025 (Rejected), B023 (Cancelled), B017/B021 (Checked In), B001–B015 (Completed), B012/B030 (No-show)
5 space statuses: Available, In Use, Under Maintenance, Temporarily Closed, Retired	SP001–SP004/SP006/SP007/SP009/SP011 (Available), SP005 (Under Maintenance), SP008/SP010 (Temporarily Closed), SP012 (Retired)
6 user roles: Student, Lecturer, Teaching Assistant, Facility Staff, Department Administrator, Facility Manager	U001–U008 (Student), U009–U013 (Lecturer), U014–U016 (TA), U017–U018 (Facility Staff), U019 (Dept. Admin), U020 (Facility Manager)
4 maintenance statuses: Pending, In Progress, Completed, Cancelled	M002 (Pending), M001/M008 (In Progress), M003–M007/M009/M012 (Completed), M010/M011 (Cancelled)
3 account statuses: Active, Inactive, Suspended	U007 (Inactive), U008 (Suspended), all others (Active)
Optional relationship cases (1:0..1)	Bookings without approval (B019, B028), bookings without usage session (B012, B016, B018–B020, B022–B030)
Edge cases	No-show with approval but no session (B012/B030), ongoing sessions with NULL <code>actual_end_time</code> (S015/S016), rejected bookings with recorded reasons (B024/B025), maintenance on retired space (M011)

§3.3 Query Design

Twenty-four queries were designed to address operational, analytical, and reporting needs across all user roles. The complete set is provided in the project deliverables (output/07-query-design-G08.sql).

§3.3.1 Query Summary

The following table lists a representative subset of the 24 queries:

Query #	Business Question	Target User
Q01	Which spaces are scheduled to be used in the future?	Facility Staff/Manager
Q05	Which spaces are used most frequently?	Facility Manager
Q06	How many maintenance tasks is each staff member responsible for?	Facility Manager
Q13	Which pending requests have events starting soonest?	Facility Staff
Q14	What is the average actual session duration for each space type?	Facility Manager
Q16	What is the no-show rate for each user?	Facility Manager/Dept. Admin
Q17	Which departments submit the most requests, and what is their approval rate?	Facility Manager/Dept. Admin
Q20	Which maintenance issues have been open for more than 7 days?	Facility Manager
Q22	Which spaces have both high bookings and frequent maintenance?	Facility Manager
Q24	What is the expected occupancy rate for upcoming approved bookings?	Facility Staff/Manager

§3.3.2 Representative Query Results

The following queries demonstrate key SQL techniques used across the 24-query set.

3.3.2.1 Aggregation with GROUP BY and CASE expressions. Query 16 calculates the no-show rate for each user by combining COUNT, conditional SUM with CASE, and CAST for percentage formatting. The HAVING clause filters to users with at least one booking, and NULLIF prevents division by zero.

```

1  -- Q16: No-show rate per user
2  SELECT
3      u.user_id,
4      u.full_name,
5      COUNT(br.booking_id) AS total_bookings,
6      SUM(CASE WHEN br.status = 'no_show'
7            THEN 1 ELSE 0 END) AS no_show_count,
8      CAST(
9          100.0
10         * SUM(CASE WHEN br.status = 'no_show'
11               THEN 1 ELSE 0 END)
12         / NULLIF(COUNT(br.booking_id), 0)
13         AS DECIMAL(5,2)
14     ) AS no_show_rate_pct
15 FROM USERS u
16 JOIN BOOKING_REQUEST br ON u.user_id = br.user_id
17 GROUP BY u.user_id, u.full_name
18 HAVING COUNT(br.booking_id) > 0
19 ORDER BY no_show_rate_pct DESC;

```

3.3.2.2 T-SQL features: GETDATE, DATEDIFF, and TOP WITH TIES. Query 05 uses TOP 1 WITH TIES to return all spaces that share the highest usage frequency, rather than arbitrarily picking one. The COUNT aggregation with GROUP BY computes usage frequency, and the IN predicate filters to bookings that were actually used (approved, checked in, or completed).

```

1 -- Q05: Most frequently used spaces
2 SELECT TOP 1 WITH TIES
3     s.space_code,
4     s.space_name,
5     COUNT(br.booking_id) AS usage_frequency
6 FROM SPACES s
7 JOIN BOOKING_REQUEST br
8     ON s.space_code = br.space_code
9 WHERE br.status IN (
10     'approved', 'checked_in', 'completed'
11 )
12 GROUP BY s.space_code, s.space_name
13 ORDER BY usage_frequency DESC;

```

3.3.2.3 Cross-table business logic with DATEDIFF and GETDATE. Query 20 identifies overdue maintenance tasks by joining MAINTENANCE_RECORD with SPACES and USERS (for both the reporter and assigned staff), then using DATEDIFF(DAY, ...) with GETDATE() to compute how long each issue has been open. Only tasks with status 'open' or 'in_progress' that exceed 7 days are returned.

```

1 -- Q20: Maintenance issues open for more than 7 days
2 SELECT
3     m.maintenance_id,
4     s.space_name,
5     m.problem_description,
6     u_rep.full_name AS reporter_name,
7     u_asgn.full_name AS assigned_staff_name,
8     DATEDIFF(DAY, m.start_time, GETDATE())
9     AS days_open
10 FROM MAINTENANCE_RECORD m
11 JOIN SPACES s
12     ON m.space_code = s.space_code
13 JOIN USERS u_rep
14     ON m.reporter_user_id = u_rep.user_id
15 LEFT JOIN USERS u_asgn
16     ON m.assigned_staff_user_id = u_asgn.user_id
17 WHERE m.status IN ('open', 'in_progress')
18 AND DATEDIFF(DAY, m.start_time, GETDATE()) > 7;

```

§4 Conclusion

§4.1 Project Summary

This project designed and implemented a relational database system for managing campus space booking, approval, usage sessions, and maintenance for the School of Computer Science. The system replaces a manual spreadsheet and email process with a structured database encompassing 7 entities—USER, SPACE, FACILITY, BOOKING_REQUEST,

`BOOKING_APPROVAL`, `USAGE_SESSION`, and `MAINTENANCE_RECORD`—connected by 11 relationships that model the full booking lifecycle from request submission through approval, check-in, and completion. The database enforces business constraints at the data level, including overlap prevention, space availability checks, and historical data preservation. All deliverables across 7 sections were developed through iterative refinement using a structured Agent/Skill/Evaluation framework with human-in-the-loop evaluation, where each section’s output was scored against a fixed rubric and progressively improved until quality targets were met.

§4.2 Key Achievements

- **Database design:** Complete business requirements analysis identifying 6 actor roles, conceptual ERD with proper cardinalities and participation constraints, logical schema with 7 relations in Third Normal Form, and thorough design validation—covering the full booking-maintenance lifecycle from request to resolution.
- **SQL implementation:** Technically sound DDL with appropriate primary keys, 11 foreign keys, `CHECK` constraints for domain validation, `UNIQUE` constraints enforcing 1:0..1 cardinalities on `BOOKING_APPROVAL` and `USAGE_SESSION`, triggers for cross-table business rules, and realistic sample data covering all status values (7 booking statuses, 5 space statuses, 4 maintenance statuses) and edge cases. A total of 24 business queries were implemented in T-SQL spanning single-table retrieval, multi-table joins, aggregation, subqueries, and window functions.
- **AI agent improvement:** Demonstrated iterative refinement across 7 sections with measurable score improvements (e.g., Section 01: 8→10, Section 04: 7.5→9, Section 05: 10/10). The process evolved from basic prompting to a structured Agent/Skill/Evaluation framework, where reusable global knowledge was maintained in `AGENT.md`, section-specific methodology was encoded in skill files, and outputs were evaluated against fixed rubrics with human-supervised learning guiding each refinement round.

§4.3 Future Work

- **Notification system:** Implement automated notifications for booking confirmations, approval decisions, and maintenance status updates via email or in-app alerts.
- **Authentication and access control:** Add authentication and role-based access control at the application layer to enforce permission policies (e.g., only facility staff may approve bookings or perform check-in operations).
- **Analytics dashboard:** Develop a visual dashboard for space utilization statistics, booking trends by department or time period, and maintenance frequency analysis.
- **Temporal utilization queries:** Extend the query suite with utilization rate calculations (actual usage hours divided by available hours per space) to support data-driven space allocation decisions.